



PROGRAMAÇÃO I

Aula 05_1 Java

Vinicius Bischoff

Existem formas importantes de programação de computadores:

- A programação estruturada (PE)
- A programação orientada a objetos (POO)

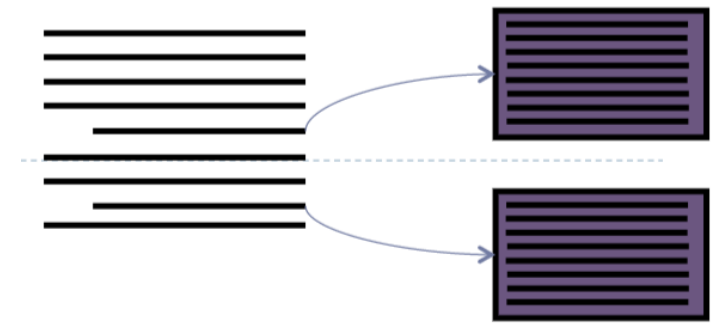
Programação estruturada:

- Construir programas usando as 3 estruturas de controle (sequência, seleção e repetição).
- Foi o que se estudamos anteriormente, usando a linguagem Python.
- Programação orientada a objetos:
 - Preocupa-se com os dados e com as operações que podem ser executadas sobre estes dados que o programa irá tratar.
 - Classe: uma estrutura de dados e suas operações associadas.
 - Programa: composto de uma ou mais classes.

POO – Programação Orientada a Objetos

> Sinônimo: paradigma procedural

> Uso de subprogramação



1. Agrupamento de c  digo permitindo a cria  o de a   es complexas
 2. Atribui  o de um nome para essas a   es complexas
 3. Chamada a essas a   es complexas de qualquer ponto do programa
-   Em Java, essas a   es complexas s  o denominadas **m  todos**
 -   Outras linguagens usam termos como procedimento, sub-rotina e **fun   o**

Paradigma procedimental

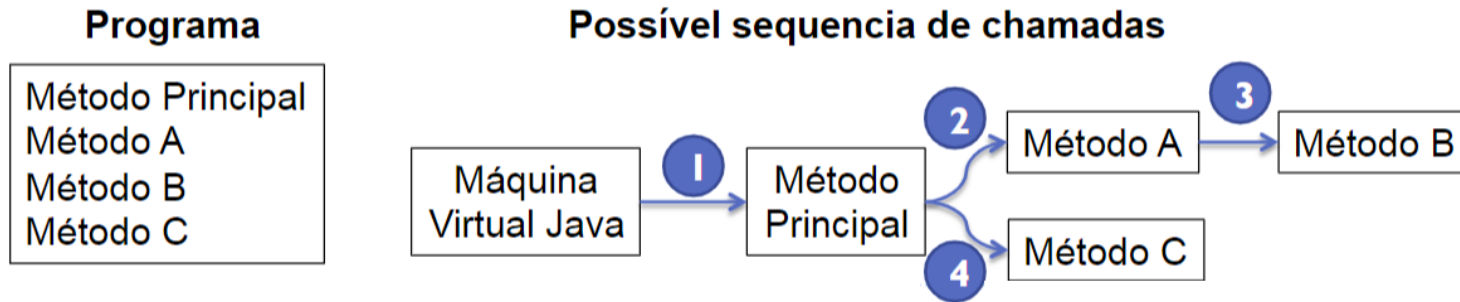
- › Antes: um programa gigante



- › Depois: vários programas menores

Dividir para conquistar

- › O programa tem início em um método principal (no caso do Java é o método **main**)
- › O método **principal** chama outros métodos
- › Estes métodos podem chamar outros métodos, **sucessivamente**
- › Ao fim da execução de um método, o programa retorna para a instrução seguinte à da chamada ao método



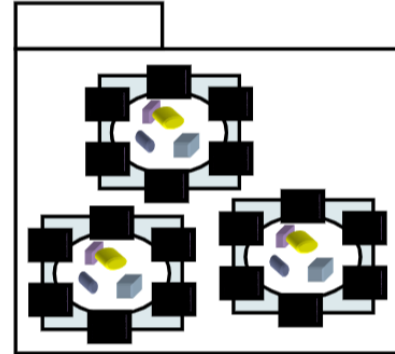
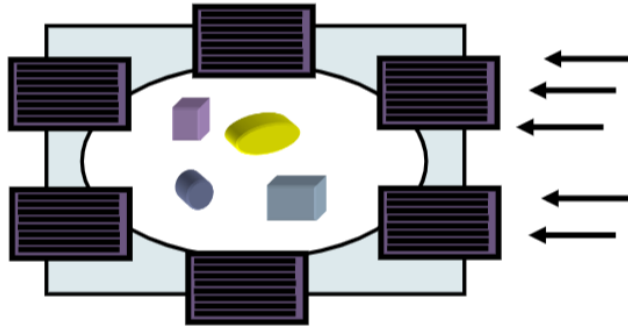
Fluxo de execução

Classes de objetos

- Agrupamento de métodos afins

Pacotes de classes

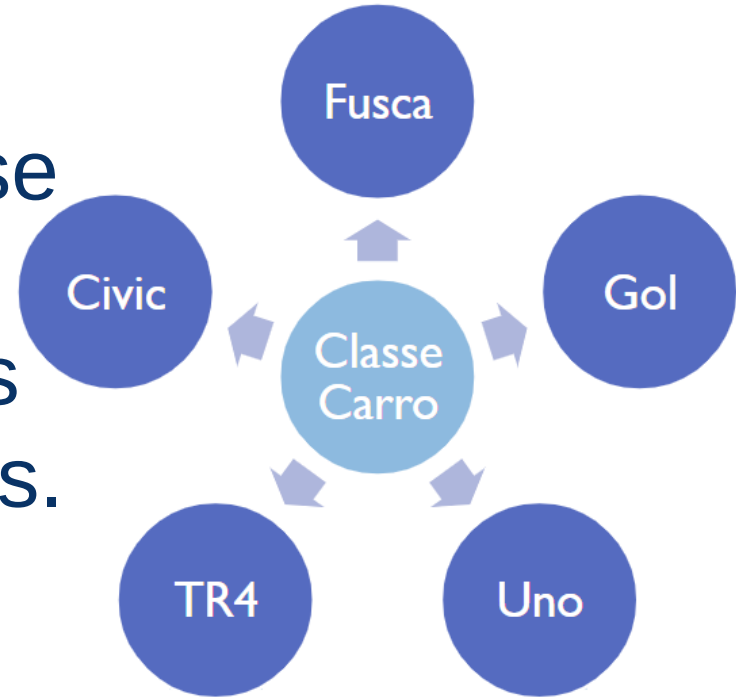
- Agrupamento de classes afins
- Representam bibliotecas de apoio



Paradigma orientado a objetos (OO)

Classes x Objetos

Uma classe é como se fosse uma fôrma, capaz de produzir (instanciar) objetos com características distintas.



Paradigma orientado a objetos (OO)

Objetos

Definição

- Um objeto é a representação computacional de um elemento ou processo do mundo real.
- Cada objeto possui suas **características** e seu **comportamento**

Exemplos de Objetos:

- cadeira - mesa - caneta - lápis
- carro – piloto – venda - mercadoria – cliente
- aula – programa - computador – aluno - avião

Paradigma orientado a objetos (OO)

Características de Objetos

Definição

- Uma característica descreve uma propriedade de um objeto, ou seja, algum elemento que descreva o objeto.
- Cada característica é chamada de atributo do objeto.

Exemplo de características do objeto carro

- Cor
- Marca
- Número de portas
- Ano de fabricação
- Tipo de combustível

Paradigma orientado a objetos (OO)

Comportamentos de Objetos

Definição

- Um comportamento representa uma ação ou resposta de um objeto a uma ação do mundo real.
- Cada comportamento é chamado de **método** do objeto.

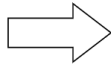
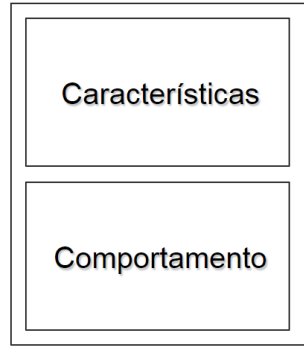
Exemplo de características do objeto carro

- Acelerar
- Frear
- Virar para direita
- Virar para esquerda

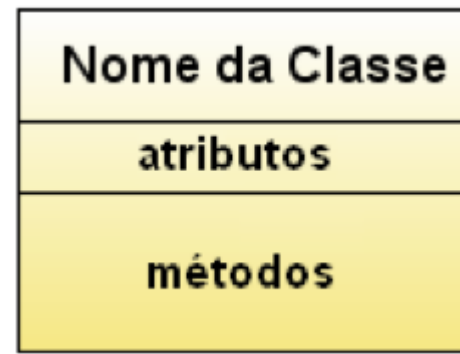
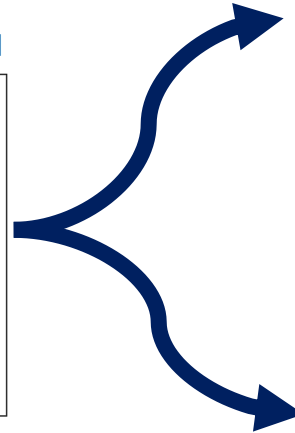
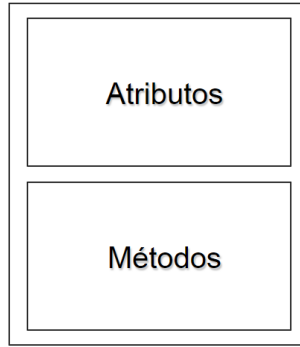
Paradigma orientado a objetos (OO)

Mapeamento de Objetos

Objeto no Mundo Real



Objeto Computacional

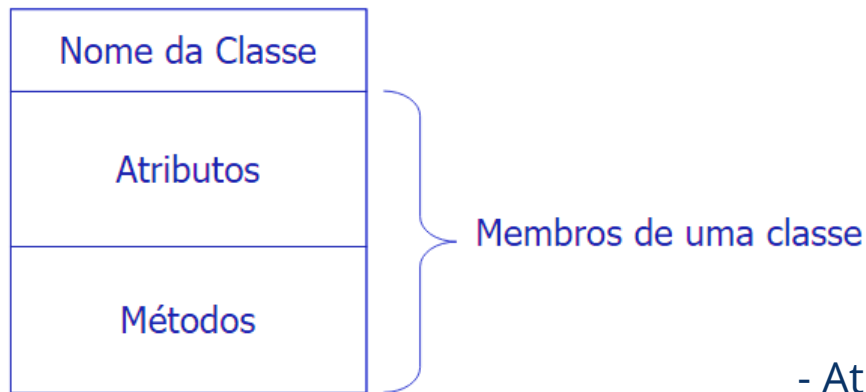


exemplo de classe
representada em uml

```
1. public class NovaClasse {  
2.     // aqui definiremos suas propriedades  
3. }
```

Paradigma orientado a objetos (OO)

Mapeamento de Objetos



Sintaxe de uma classe em java

```
modificadores class nome da classe {  
    atributos  
    métodos  
}
```

```
1. public class Pessoa {  
2.     String nome;  
3.     String genero;  
4.     public void jogar(){  
5.         System.out.println("Estou jogando bola");  
6.     }  
7. }
```

- Atributo

Representam os dados que cada objeto da classe irá guardar. É a característica do objeto (conhecimento) -

- Métodos

São funções ou ações que um objeto pode executar sobre os seus atributos ou para propósitos gerais (comportamento)

Paradigma orientado a objetos (OO)

Cada classe define:

- Uma **estrutura de dados**: constituída por um conjunto de **campos** que caracterizam um particular tipo de dado.
- Operações possíveis: um conjunto de funções, denominadas **métodos**, que implementam as operações possíveis sobre os dados armazenados nos campos

Notação gráfica para representar uma classe:

Nome da Classe
Campos: estrutura de dados da classe
Métodos: funções que implementam as operações possíveis.

Exemplo:

Retangulo
– int base – int altura
+ void setBase(int b) + void setAltura(int a) + int area()

Notar que, em geral, os campos são **privados** (indicado pelo sinal –) e os métodos são **públicos** (indicado pelo sinal +).

Paradigma orientado a objetos (OO)

Uma classe pode ter campos e métodos privados ou públicos.

- Campos: devem ser privados (para garantir a consistência dos objetos).
- Campos públicos: no caso de constantes (valor não pode ser alterado).
- Métodos: em geral, devem ser públicos (API da classe)
- Métodos privados: quando são internos à classe (não estão na API)

Em geral, os campos e métodos de uma classe podem ser:

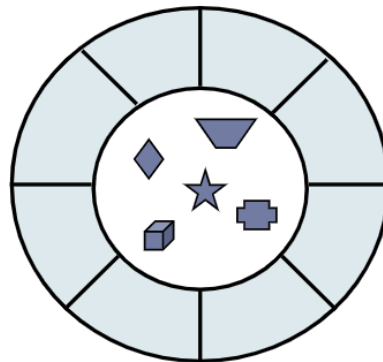
Nível de acesso				
Modificador	Classe	Pacote	Subclasse	Todas
public	S	S	S	S
protected	S	S	S	N
(package)	S	S	N	N
private	S	N	N	N

Private (-)
Public (+)
Protected (#)

Encapsulamento

Atributos e Métodos

- Os métodos formam uma “cerca” em torno dos atributos
- Os atributos não devem ser manipulados diretamente
- **Os atributos somente devem ser alterados ou consultados através dos métodos do objeto.**

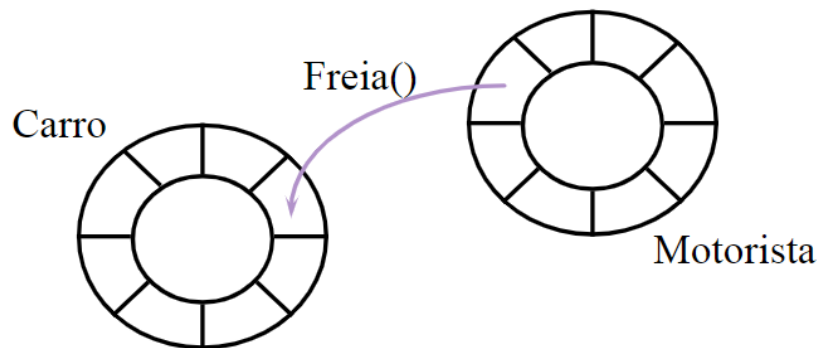


Paradigma orientado a objetos (OO)

Chamada de métodos

Colaboração

- Um programa OO é um conjunto de objetos que colaboram entre si para a solução de um problema
- Objetos colaboram através de chamadas de métodos uns dos outros



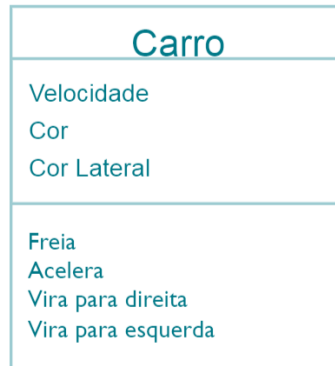
Paradigma orientado a objetos (OO)

Classes

- A classe descreve as características e comportamento de um conjunto de objetos.
 - Em Java, cada objeto pertence a uma única classe
 - O objeto possuirá os atributos e métodos definidos na classe
 - O objeto é chamado de instância de sua classe
 - A classe é o bloco básico para a construção de programas OO.

Paradigma orientado a objetos (OO)

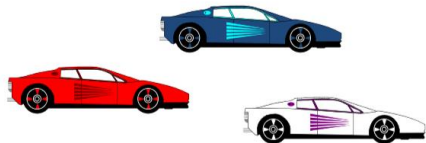
Exemplo de Classe



```
public class Carro {  
    private int velocidade;  
  
    public void acelera() {  
        velocidade++;  
    }  
  
    public void freia() {  
        velocidade--;  
    }  
}
```

Atributos (características) são **variáveis globais** acessíveis por todos os métodos da classe

Métodos (comportamentos)



Paradigma orientado a objetos (OO)

Criação de Objetos

A classe é responsável pela criação de seus objetos via método construtor

- Mesmo nome da classe
- Sem tipo de retorno

```
public Carro(int velocidadeInicial) {  
    velocidade = velocidadeInicial;  
}
```

Paradigma orientado a objetos (OO)

Criação de Objetos

- Objetos devem ser instanciados antes de utilizados
- O comando new instancia um objeto, chama o seu construtor

Exemplo:

```
Carro fusca = new Carro(10);  
Carro bmw = new Carro(15);  
fusca.freia();  
bmw.acelera();  
fusca = bmw;
```

Qual a velocidade de cada
carro em cada momento?

O que acontece aqui?

Paradigma orientado a objetos (OO)

Convenção de nomes em um programa Java

- Nome de Classe: Escrito com a primeira letra em maiúscula. Se o nome for composto de várias palavras, a primeira letra de cada palavra deve ser maiúscula (Exemplos: Retangulo, TrataRet, Tempo, ExeTempo).
- Nome de Campo ou de Método: Deve começar com uma palavra escrita em minúsculas. Se contiver outras palavras, a primeira letra de cada palavra deve ser maiúscula (Exemplos: base, altura, area, converter).

Construtores têm o **mesmo nome** da classe.
Nomes de métodos são sempre **seguidos por parênteses**.
Por exemplo: `area()`, `converter()`, `Tempo()`.

Paradigma orientado a objetos (OO)

Convenção de nomes em um programa Java

- Em Java, um construtor é um método especial que é chamado automaticamente quando um objeto de uma classe é criado.
- O objetivo do construtor é inicializar os membros da classe e preparar o objeto para uso. Aqui está um exemplo de um construtor em Java:

Paradigma orientado a objetos (OO)

Convenção de nomes em um programa Java

```
public class Carro {  
    private String marca;  
    private String modelo;  
    private int ano;  
    // Construtor da classe Carro  
    public Carro(String marca, String modelo, int ano) {  
        this.marca = marca;  
        this.modelo = modelo;  
        this.ano = ano;  
    }  
    // Método que retorna a marca do carro  
    public String getMarca() {  
        return marca;  
    }  
    // Método que retorna o modelo do carro  
    public String getModelo() {  
        return modelo;  
    }  
    // Método que retorna o ano do carro  
    public int getAno() {  
        return ano;  
    }  
}
```

O construtor da classe Carro é definido como public Carro(String marca, String modelo, int ano).

Este construtor recebe três parâmetros (marca, modelo e ano) e inicializa os membros correspondentes da classe usando a palavra-chave this.

O método getMarca(), getModelo() e getAno() são métodos de acesso que permitem que você obtenha os valores das variáveis marca, modelo e ano, respectivamente, de um objeto Carro criado com esse construtor.

Convenção de nomes em um programa Java

Java API

- Um dos pontos fortes da linguagem Java é o seu conjunto de classes primitivas: a Java API (Applications Programming Interface).
- A Java API está organizada em pastas (denominadas pacotes) e deve ser constantemente consultada para obter informações sobre os campos e métodos de cada uma dessas classes primitivas.

Paradigma orientado a objetos (OO)

Na maioria dos casos, para o atributo, teremos um método “getter” (retorna a informação) e um método “setter” (atribui informação).

Ex: getNome(), setNome(String nome)

- Procurem colocar sempre o atributo e o método em minúsculo (o java é case-sensitive)
- Para atributos/métodos com nome composto, a primeira palavra em minúsculo e, para as demais, a primeira letra em maiúsculo: numeroContrato, dataNascimento,...

Paradigma orientado a objetos (OO)

Classe

Pessoa
+nome: String -altura: double -peso: double
+ getNome(): String + setNome(): void + getAltura(): double + setAltura(): void + getPeso(): double + setPeso(): void - calculaIMC(): double + getIMC(): double

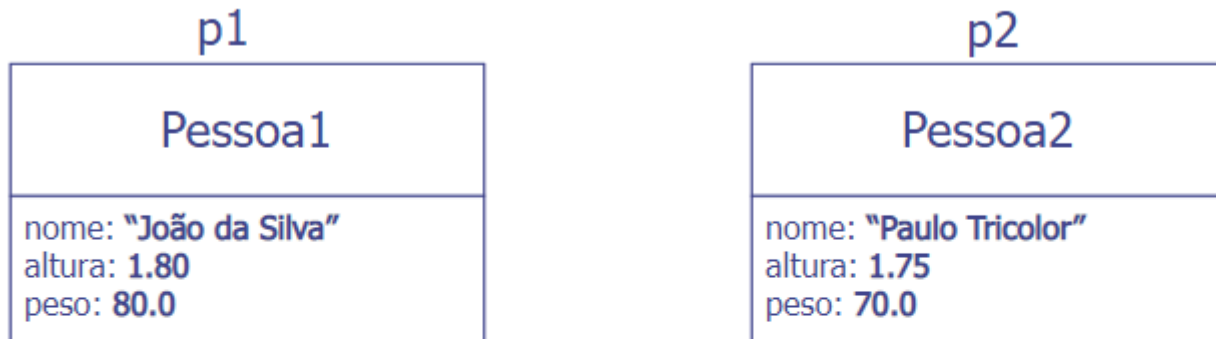
Em java teremos:

```
public class Pessoa {  
    public String nome;  
    private double altura;  
    private double peso;  
  
    public String getNome() {}  
    public void setNome() {}  
    public double getAltura() {}  
    public void setAltura() {}  
    public double getPeso() {}  
    public void setPeso() {}  
    private double calculaIMC() {}  
    public double getIMC() {}  
}
```

A estrutura de uma classe em java
está definida!
Precisaremos implementar os métodos!

Paradigma orientado a objetos (OO)

- Diagrama de Objetos



EM JAVA TEREMOS:

```
Pessoa p1 = new Pessoa()  
p1.setNome("João da Silva");  
p1.setAltura(1.80);  
p1.setPeso(80.0);
```

```
Pessoa p2 = new Pessoa()  
p2.setNome("Paulo Tricolor");  
p2.setAltura(1.75);  
p2.setPeso(70.0);
```

Paradigma orientado a objetos (OO)

Toda classe possui um método especial: construtor. Para que serve?

O construtor é usado para construir objetos da classe.

Por que é necessário criar objetos de uma classe?

Para utilizar os recursos da classe.

Quais são os recursos utilizáveis de uma classe?

Seus métodos (parte pública da classe).

Pessoa	← nome da classe
- double altura - double peso	← campos
+ void setAltura(double a) + void setPeso(double p) + double calcularIMC()	← métodos

```
Pessoa p = new Pessoa();  
p.setAltura(1.85);  
p.setPeso(88.0);  
System.out.println("IMC = " + p.calcularIMC());
```

Paradigma orientado a objetos (OO)

Convenção de nomes em um programa Java

Recursos disponíveis em Java para construir métodos

- Declarações: linhas de código terminadas por ponto-e-vírgula usadas para estabelecer os tipos dos identificadores usados no programa. Exemplos:

```
int i;  
int j = 10;  
Pessoa p;  
Circulo c = new Circulo();
```

- Instruções: linhas de código terminadas por ponto-e-vírgula usadas para estabelecer ações: comandos de atribuição ou chamadas de métodos. Exemplos:

```
i = 0;  
int v = v1 + v2 + v3;  
Pessoa p = new Pessoa();  
System.out.printf("IMC = %.2f\n", p.calcularIMC());
```

Paradigma orientado a objetos (OO)

Convenção de nomes em um programa Java

- Blocos: conjuntos de declarações e instruções delimitados por chaves.

Exemplo:

```
{  
    int valor1, valor2;  
    valor1 = v1 + v2 + v3;  
    valor2 = a + b;  
}
```

- Estruturas de controle: controlam como as instruções de um bloco devem ser executadas. Exemplos:

```
if (i < j)  
{  
    i = j;  
}  
else  
{  
    j = 0;  
}
```

```
for (int i = 0; i < 10; i++)  
{  
    k = k + Math.sqrt(i);  
}
```

```
while (i < 5)  
{  
    System.out.println(2*i);  
    i++;  
}
```

Paradigma orientado a objetos (OO)

Tipos primitivos de dados

- Em Java existem 8 tipos primitivos separados em quatro categorias:

Categoria	Tipos primitivos
Lógico	boolean
Textual	char
Inteiro	byte, short, int, long
Ponto flutuante	double, float

- Para cada tipo primitivo há uma classe correspondente no pacote java.lang. Por exemplo: Integer (para o tipo int), Double (para o tipo double), Float (para o tipo float), ...
- Essas classes fornecem métodos para processamento dos valores do tipo primitivo. Exemplos:

```
int d = Integer.valueOf("234");  
String s = String.valueOf(d);
```

O tipo boolean

- Dois valores apenas para o tipo boolean: **true** e **false**

```
boolean b = false;  
boolean deNovo;  
deNovo = (x < 10);
```

- Operadores lógicos e booleanos

Operador	Nome
&&	conjunção lógica
&	conjunção booleana
	disjunção lógica
	disjunção booleana
^	disjunção lógica exclusiva
!	negação lógica

Sejam:

```
int x = 1;  
int y = 4;
```

```
(x == 1) && (y >= 5)    false  
(x == 5) & (y >= 1)     false  
(x == 1) || (y >= 5)    true  
(x == 1) | (y >= 5)     true  
(x == 1) ^ (y <= 5)     false  
! ( y >= 5 )            true
```


○ Operadores aritméticos

Precedência	Operador	Operação	Exemplo	Resultado
1	+	Adição	7 + 2	9
1	-	Subtração	7 - 2	5
2	*	Multiplicação	7 * 2	14
2	/	Divisão	7 / 2	3 (int) ou 3.5 (float)
2	%	Resto	7 % 2	1

Um erro muito comum é utilizar = como operador de igualdade. Lembrar que = é o operador de **atribuição**.

○ Operadores de igualdade

Operador	Operação	Exemplo	Resultado
==	é igual a	7 == 2	falso
!=	não é igual a	7 != 2	verdadeiro

○ Operadores de incremento e de decremento

Operador	Nome	Exemplo	Significado
++	pré-incremento	++x	Incrementa x de 1 e então usa o novo valor de x na expressão onde x reside.
++	pós-incremento	x++	Usa o valor de x na expressão onde x reside e depois incrementa x de 1.
--	pré-decremento	--x	Decrementa x de 1 e então usa o novo valor de x na expressão onde x reside.
--	pós-decremento	x--	Usa o valor de x na expressão onde x reside e depois decrementa x de 1.

○ Operadores relacionais

Operador	Operação	Exemplo	Resultado
>	é maior que	7 > 2	verdadeiro
<	é menor que	7 < 2	falso
>=	é maior que ou igual a	7 >= 2	verdadeiro
<=	é menor que ou igual a	7 <= 2	falso

Não pode haver espaço entre os símbolos dos operadores:

==

!=

>=

<=

○ Operadores de incremento e de decremento

Operador	Nome	Exemplo	Significado
++	pré-incremento	++x	Incrementa x de 1 e então usa o novo valor de x na expressão onde x reside.
++	pós-incremento	x++	Usa o valor de x na expressão onde x reside e depois incrementa x de 1.
--	pré-decremento	--x	Decrementa x de 1 e então usa o novo valor de x na expressão onde x reside.
--	pós-decremento	x--	Usa o valor de x na expressão onde x reside e depois decrementa x de 1.

○ Operadores relacionais

Operador	Operação	Exemplo	Resultado
>	é maior que	7 > 2	verdadeiro
<	é menor que	7 < 2	falso
>=	é maior que ou igual a	7 >= 2	verdadeiro
<=	é menor que ou igual a	7 <= 2	falso

Não pode haver espaço entre os símbolos dos operadores:

==

!=

>=

<=

Tipos char e a classe String

- Um valor do tipo char representa um único caractere(Unicode de 16 bits) e deve ser escrito entre apóstrofos. Por exemplo: 'a'.
- String não é um tipo primitivo e sim uma classe. Objetos dessa classe representam textos, ou seja, sequências de caracteres. Strings devem ser escritos entre aspas. Exemplo: "Linguagem Java".

Exemplos:

- `char x = 'c';` a letra c
- `char nl = '\n';` caractere de 'nova linha'
- `char fi = '\u03A6';` a letra Φ
- `String sigla = "PC2";`
- `String nome = "Programação de Computadores II\n";`

Os tipos inteiros (byte, short, int, long)

- **int** é o tipo padrão para números inteiros.
- Os números inteiros podem ser utilizados em três formatos: decimal, octal e hexadecimal.
- Exemplos:

Valor	Notação	Observação
56	decimal	Notação usual
070	octal	Iniciado por 0 (zero). Notar que: $(70)_8 = (56)_{10}$
0x38	hexadecimal	Iniciado por 0x (zero xis). Notar que: $(38)_{16} = (56)_{10}$
56L	longo	Terminado por L ou l (valor inteiro do tipo long)

Tipo	Bits	Intervalo de valores
byte	8	$[-128, 127]$
short	16	$[-32768, 32767]$
int	32	$[-2147483648, 2147483647]$
long	64	$[-9223372036854775808, 9223372036854775807]$

Os tipos de ponto flutuante (float,double)

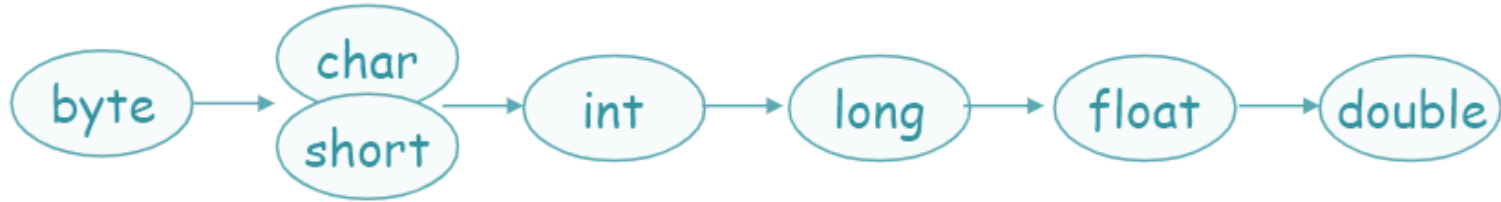
- **double** é o tipo padrão para números de ponto flutuante.
- Valores do tipo **float** exigem a presença da letra F (ou f) ao final.
- Exemplos:

Valor	Observação
2.34	Valor de ponto flutuante escrito na notação usual
0.234E1	Valor de ponto flutuante com expoente positivo
23.4E-1	Valor de ponto flutuante com expoente negativo
2.34F	Valor de ponto flutuante do tipo float
2.34D	Valor de ponto flutuante do tipo double
2.34	Valor de ponto flutuante do tipo double

Tipo	Bits	Intervalo de valores
float	32	$[-3.40292347E+38, 3.40292347E+38]$
double	64	$[-1.79769313486231570E+308, 1.79769313486231570E+308]$

Conversão de tipos

- Uma conversão ocorre ao se atribuir um valor de um tipo a uma variável de outro tipo.
- Se o tipo da variável for maior do que o tipo do valor a ser atribuído, a conversão é automática e feita no momento da atribuição.



- Do contrário (tipo da variável < tipo do valor), deve haver uma conversão explícita do tipo do valor (cast)

Variáveis de referência (ponteiros)

- Em Java, uma variável pode representar um valor de um dos tipos primitivos ou um objeto de uma classe (nova ou da API).
- Uma variável que representa um objeto é conhecida como variável de referência (também chamada de ponteiro).
- Exemplos:

```
public class MinhaData
{
    private int dia;
    private int mes;
    private int ano;
    ...
}
```

Observe que **d** é uma variável do tipo **MinhaData**, ou seja, representa um objeto da classe **MinhaData**. Portanto, **d** é uma variável de referência (um **ponteiro**).

```
public class Teste
{
    MinhaData d = new MinhaData();
    d.setDia(01);
    d.setMes(09);
    d.setAno(2013);
    ...
}
```


Obrigado.

